Operating Systems and Parallel Computing

Lecture 12

Prof. Matteo Turilli

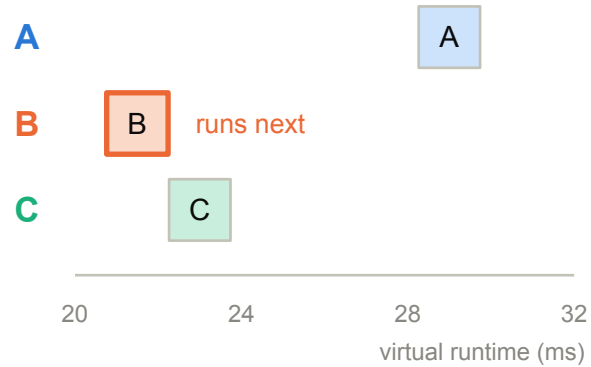
Linux Took a Different Route

- In 2003, Linux 2.6 introduced the O(1) scheduler, a close relative of MLFQ, with 140 priority levels and a queue for each level.
- Linux calls what it schedules a **task**: a process, or one thread of a process. OSTEP, and our animations, call them **jobs**.
- It always ran the first task in the highest-priority non-empty queue. Finding that task took the same time however many tasks were waiting: hence the name O(1).
- Each task sat in one of two sets of queues, active or expired. A task that used up its time slice moved to expired; when active was empty, the two sets swapped.
- Like MLFQ, O(1) guessed from past behavior which tasks were interactive: tasks that often slept got a priority bonus, those that used a lot of CPU got a penalty.
- The guesses were complex and often wrong.
- Since 2007, Linux no longer guesses: each task gets a fair share of the CPU, set by a `nice` value.



CFS: The Task That Is Behind Runs Next

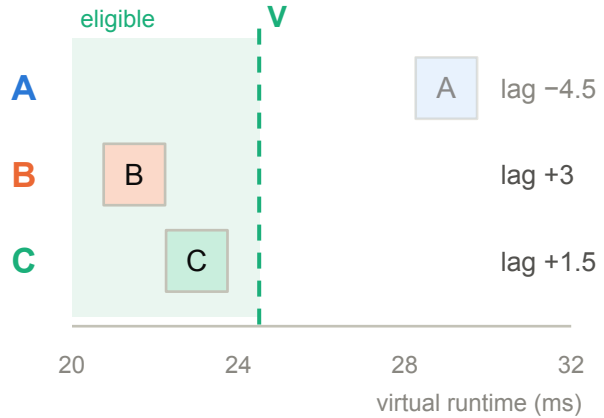
- Completely Fair Scheduler (CFS)
- CFS keeps a score for each task, its **virtual runtime**.
- While a task runs, its score grows with the CPU time it gets: 1 ms of CPU adds 1 ms (see `nice` later).
- It always runs the task with the lowest score.
- Each scheduling turn lasts 24ms divided by the number of tasks ready to run, and at least 3ms.
- The score is a position, not a total: a new task starts at the lowest score, not at 0. A waking task keeps its old score, but at most 12ms below the lowest.
- A task that joins interrupts the running one only if it is more than 4ms behind it.
- **The weak spot**: how soon a task ran depended on special rules like these and on hard-to-tune settings.



B has the lowest score, so CFS runs B next.

EEVDF: Who Is Owed CPU Time

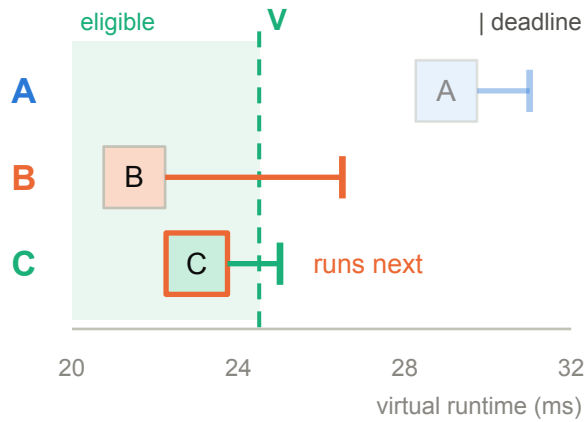
- EEVDF keeps the scores of CFS: a task's virtual runtime grows with its CPU time.
- Adds four ideas: **V**, **lag**, **slice**, and **virtual deadline**
- **V**: the average score of the tasks sharing this core, i.e., where a fair scheduler would have every task now.
- **Lag**: V − score (at nice 0), i.e., the CPU time a task is owed. Positive: it is behind. Negative: it had more than its share.
- A task is **eligible** if its lag is zero or more, while a task ahead of V must wait.
- A new task starts at V, with lag 0: it is owed nothing for the time before it arrived.
- In our animation a task back from a sleep also starts at V. Linux lets it keep its old lag, up to a limit.



V = 24.5. B and C are behind V: they are owed time, so they are eligible. A is ahead of V: it waits. The lags add up to zero.

EEVDF: Slices and Deadlines

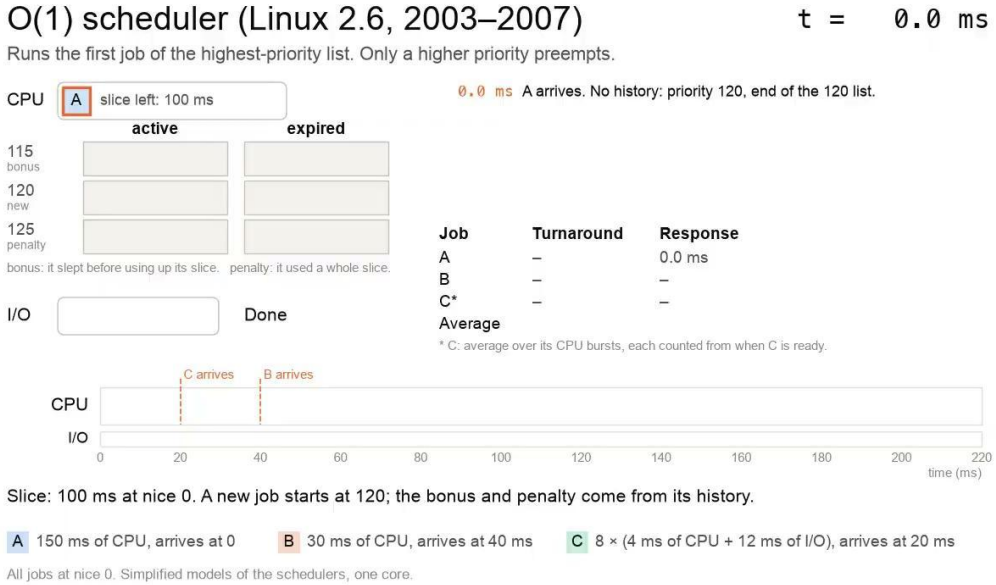
- Each task has a **slice**: the CPU time it gets each time it is picked. 3 ms by default on 8 or more CPUs; a program can set its own (0.1 to 100 ms).
- Each task's **virtual deadline** is the score it will reach when that slice is used up.
- EEVDF runs the eligible task with the earliest deadline: V decides who may run, the deadline who goes first.
- That task keeps the core until its slice is used up or it blocks; its next deadline is one slice later.
- Shorter slice: the task runs in smaller pieces, each scheduled sooner because its deadline is earlier.
- Longer slice: the task runs in bigger pieces, each scheduled later.
- No starvation: while it waits, its deadline stays put and the others' deadlines move on.
- The slice changes when a task runs, not how much CPU it gets.
- **One clear rule replaces the special cases of CFS.**



Same tasks, with slices of 2ms (A), 5ms (B) and 2ms (C). A is ahead of V and waits. B and C are eligible: C's slice is shorter, so its deadline is earlier. EEVDF runs C next.

Three Linux Schedulers in Motion

- The same three jobs under O(1), CFS and EEVDF, all at nice 0: a long job A, an interactive job C that does I/O, and a short job B, arriving at different times.
- Watch where each scheduler puts a new or waking job, and compare turnaround and response.
- Interactive version: mturilli.github.io/ospc-materials/animations/linux-schedulers.html



B Arrives at 40 ms

Task	CPU used by 40 ms	Score at 40 ms	Why
A	36 ms	36	It has been running.
C	4 ms	36	Back from I/O at 40 ms: placed at V.
B	0 ms	36	New at 40 ms: placed at V.

- All three are at V = 36, with lag 0: nobody is owed anything. From here they share the core in 3 ms turns: A 40–43 ms, B 43–46, C 46–49 (on a tie, the task queued first goes first).
- If B started at 0, it would be 36 ms behind A and would keep the core for about 36 ms. Every new program would freeze the ones already running: starting new tasks at V prevents that.
- C gets nothing back for the 12 ms it spent in I/O: a task is owed CPU time only while it waits to run, not while it sleeps.

Nice: Telling Linux What Can Wait

- Fair shares treat every task the same. But a model that retrains overnight can wait, and the editor you are typing in cannot.
- O(1) tried to guess which tasks were interactive. CFS and EEVDF do not guess: you tell them, with the **nice** value.
- Nice goes from −20 to +19, and the default is 0. A nice task leaves more CPU time to the others.
- Each nice value has a **weight** (table). Each step up divides it by about 1.25, which changes the CPU time by about 10%.
- Tasks that compete for a core share that core in proportion to their weights. Nice 0 and nice 5 on one core: 1024 / (1024 + 335) ≈ **75%** and **25%**.
- The score does the work: at nice 5, each ms of CPU adds about 3 ms to the virtual runtime (1024 / 335), so the task gets a third of the CPU time of a nice 0 task.

nice	weight
−10	9548
−5	3121
0	1024
3	526
5	335
8	172
10	110
19	15

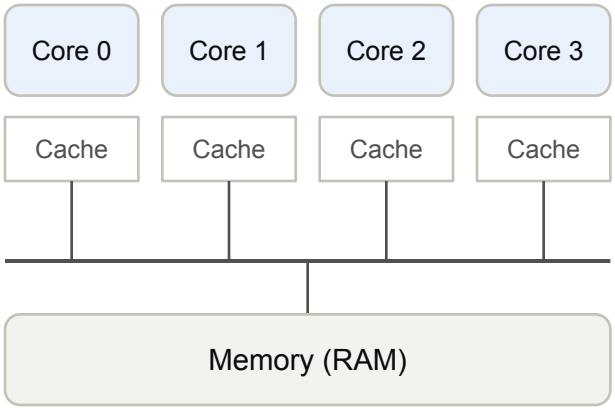
Weights from the Linux kernel (also in OSTEP ch. 9).

Nice and Priorities in Practice

- `nice -n 10 python3 train.py` starts a program with nice value 10.
- `renice -n 15 -p 4242` changes the nice value of the running process 4242.
- An ordinary user can only raise a nice value, i.e., lower the priority. Lowering a nice value needs administrator rights.
- `top` and `htop` show the nice value (**NI**) and the priority (**PR** = 20 + NI).
- Priorities matter only when processes compete for a core: alone on a core, a nice 19 process runs at full speed.
- Real-time policies, `SCHED_FIFO` and `SCHED_RR`, run before normal processes. `chrt` sets them but it requires administrator rights.
- On a shared server, start long batch jobs with `nice`, so that interactive users do not notice them.

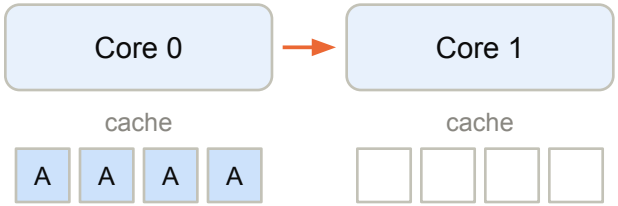
Multiple Cores

- Since about 2005, clock speeds have stopped rising: faster chips would get too hot.
- Chips add cores instead.
- A laptop has 8–16 cores and a node of our cluster has 256. A single-threaded program uses one core at a time.
- Each core has small, fast **caches** with copies of recently used data, so it waits less for RAM.
- The hardware keeps the caches **coherent**:
  - When a core writes a value, the other caches drop their old copy: no core reads a stale value.
  - Two cores can still update the same counter at once: i.e., shared data still needs locks (Lecture 5).
- The scheduler now decides **which** process runs and **on which core**.



Cache Affinity

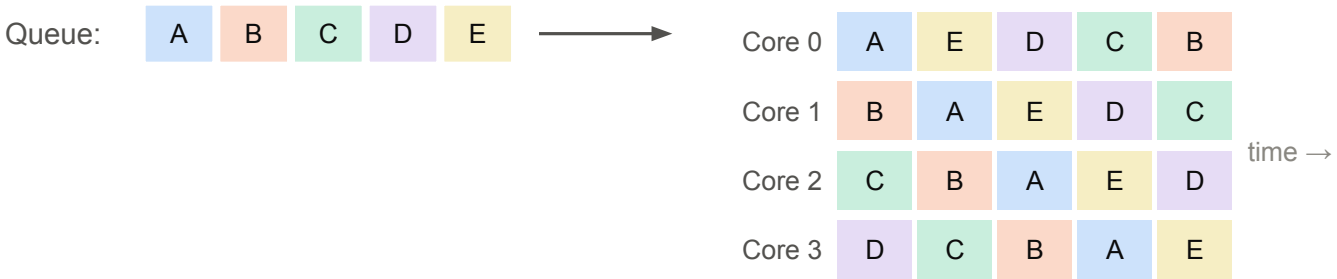
- A process that runs on a core fills that core's caches with its data.
- If it runs on the same core next time, part of its data might still be there, so it would run faster.
- If it moves to another core, it reloads its data from a shared cache or from RAM. The result is still correct, only slower.
- **Cache affinity**: the scheduler tries to keep each process on the core where it last ran.
- Servers with two sockets, like our cluster nodes, reach their own socket's RAM faster (non-uniform memory access (NUMA)).
- Linux tries to keep a process near its memory.
- You can pin your own processes: `taskset -c 0 python3 job.py` runs it only on CPU 0.



A ran on core 0: its data is in core 0's cache.
If A moves to core 1, it starts with a cold cache.

One Queue for All Cores

- **Single-queue scheduling**: every core takes its next process from one shared queue.
- It is simple, and it balances the load: no core is idle while processes wait.
- The queue is shared data, so it needs a lock: cores wait for each other to take the next process. With many cores, this does not scale.
- After each time slice, a process goes back to the queue and the next free core picks it up: processes move from core to core and lose cache affinity.



One Queue per Core

- **Multi-queue scheduling**: each core has its own queue and policy, e.g., round robin.
- There is no shared lock and processes stay on their core, so it scales and keeps cache affinity.
- The problem is **load imbalance**. See the diagram: when C finishes, A gets core 0 to itself, while B and D share core 1.
- **Migration** allows balancing the queues' load: a core with little work **steals** processes from a busier queue. Checking too often costs time, too rarely leaves cores idle.
- Linux keeps one queue per core and moves tasks between them to balance the load.

Queues A, C and B, D

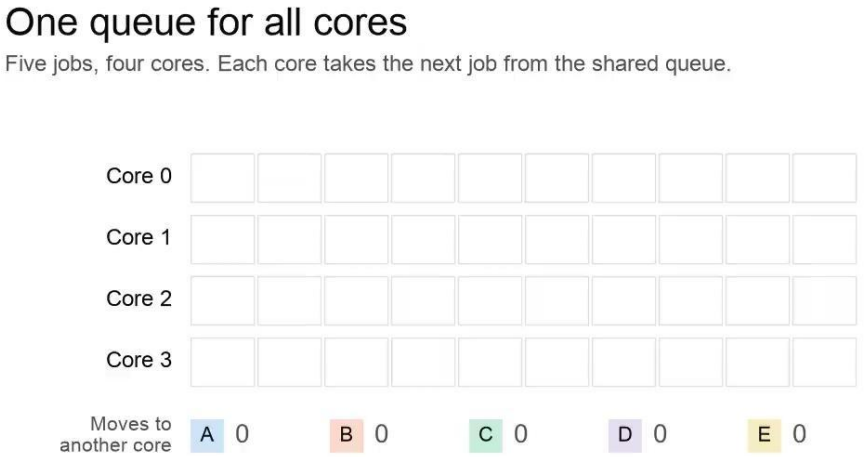


After C finishes



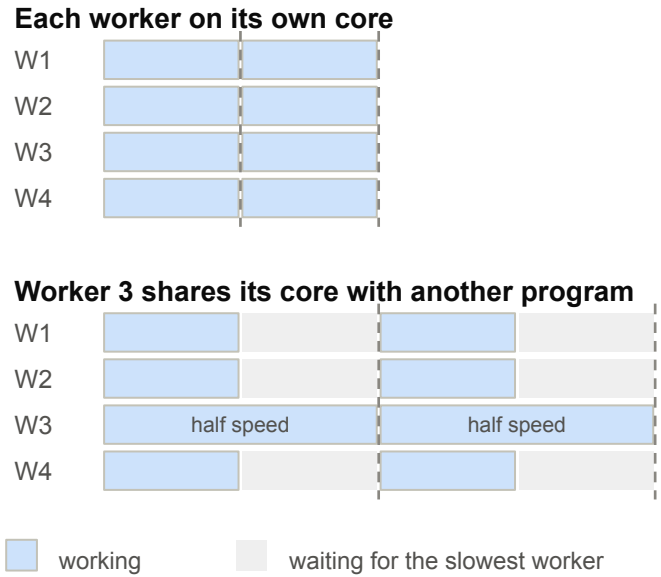
Queues and Cores in Motion

- One shared queue: Processes bounce between cores, and each move starts with a cold cache.
- One queue per core: Watch the imbalance when processes finish, and how stealing fixes it.
- Interactive version: mturilli.github.io/ospc-materials/animations/multicore-queues.html



Parallel Programs Need Their Workers Together

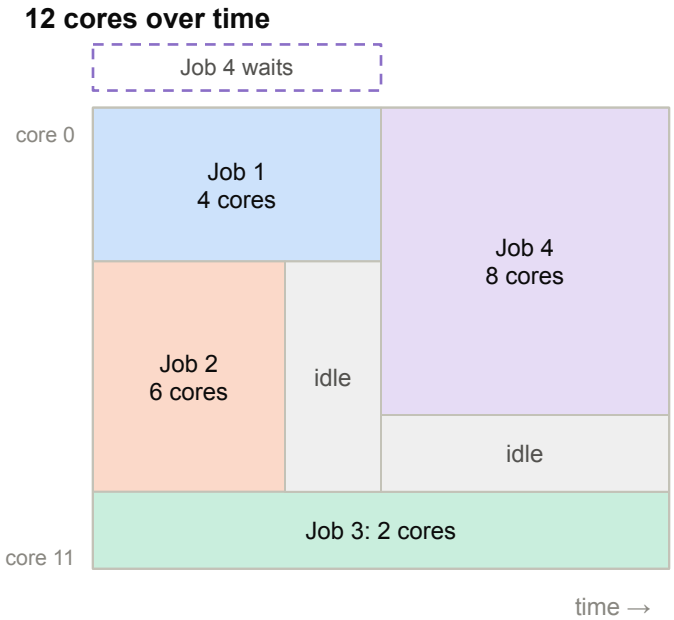
- A parallel program splits its work among **workers** that often wait for each other, e.g., at the end of each step. The program ends when the **slowest** worker ends.
- If the OS time-shares one worker with another program, that worker falls behind and all the others wait for it.
- **Gang scheduling** runs all the workers of a program at the same time, on different cores.
- More workers than cores, e.g., `n_jobs=32` on 8 cores, take turns: more switches, colder caches, no extra speed. You will measure this in Lab 1.
- `uptime` prints the **load average**: about how many processes were running or waiting to run over the last 1, 5 and 15 minutes.



Same work. One worker at half speed makes the whole program take twice as long.

Clusters Share Cores in Space

- **Space sharing** gives each job its own set of cores until it ends, so its workers never share a core with another program.
- A job that does not fit waits in a queue until enough cores are free.
- On a cluster, a **job** is a program you submit together with the resources it needs: cores, memory and time. This is not the same as the jobs in the OSTEP examples.
- Our cluster runs **Slurm**: it queues the jobs and starts each one when its cores are free. Each node has 256 cores. We will use it later in the course.



Job 4 needs 8 cores. It waits until Job 1 ends, while 6 cores sit idle.